

Student

In all the previous sections I have been querying existing tables. But how do you actually build those tables in the first place — and how do you make sure bad data never gets in?

ChatGPT • AI Assistant

CONCEPT CARD

That is exactly what **constraints** are for. A constraint is a rule enforced at the database level that prevents invalid data from being inserted, updated, or referenced. Constraints are the last line of defence — they catch what application code misses.

The five core SQL constraints:

- **PRIMARY KEY** — uniquely identifies each row, enforces NOT NULL + UNIQUE
- **FOREIGN KEY** — enforces referential integrity between two tables
- **UNIQUE** — enforces that all values in a column are distinct
- **CHECK** — enforces a custom condition on column values
- **NOT NULL** — prevents NULL values in a column

🔗 ANALOGY

Think of constraints like the validation rules in a paper form. NOT NULL is like a required field. UNIQUE is like a field that says 'no two applicants can have the same ID number'. CHECK is like a field that says 'age must be between 18 and 65'. PRIMARY KEY is the unique applicant ID printed at the top. FOREIGN KEY is the rule that says 'the department code you enter must already exist in the departments list'.

🌐 REAL WORLD APPLICATION

In production systems, constraints save you from entire categories of bugs: orphan records, duplicate entries, negative prices, invalid status values, and orders placed for non-existent customers. Removing constraints to make inserts easier is always the wrong answer — it trades short-term convenience for long-term data corruption.

Student

What scenario are we using to learn schema design?

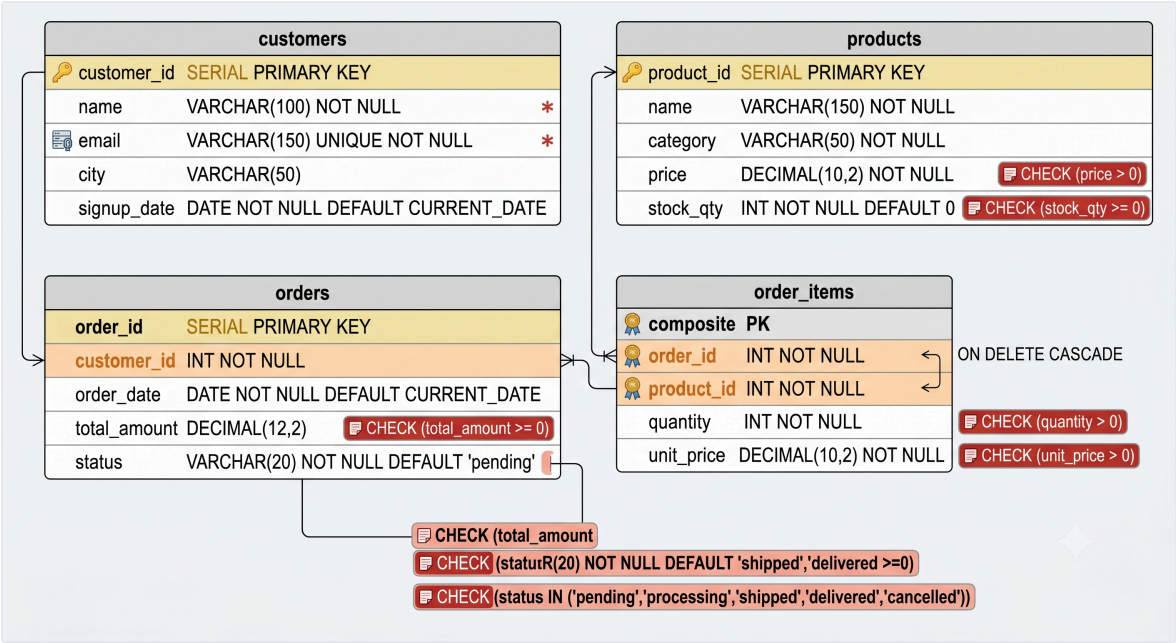
ChatGPT • AI Assistant

CONCEPT CARD

We will design an **order management system** from scratch — the same domain you have been querying since SQL 1.1, but now you will build it:

- **customers** — who is placing orders
- **products** — what can be ordered
- **orders** — the order header record
- **order_items** — individual line items within an order

By the end of this section you will have a fully constrained, production-quality schema for this system.



Order Management Schema — Fully Constrained ERD

SQL Constraints — Quick Reference

Constraint	Enforces	Allows NULL?	Multiple Per Table?	Defined On
PRIMARY KEY	Uniqueness + NOT NULL on the key column(s)	No — implicitly NOT NULL	No — exactly one per table	One column or composite (multiple columns)

Constraint	Enforces	Allows NULL?	Multiple Per Table?	Defined On
FOREIGN KEY	Referenced value must exist in parent table	Yes — unless also NOT NULL	Yes — multiple FKs allowed	Column(s) referencing another table's PK
UNIQUE	All non-NULL values in column are distinct	Yes — multiple NULLs allowed in most databases	Yes — multiple UNIQUE constraints per table	One column or composite
CHECK	Column value satisfies a boolean expression	NULL passes CHECK (NULL is not FALSE)	Yes — multiple CHECK constraints	Column or table level
NOT NULL	Column value cannot be NULL	No — that is the entire point	Yes — one per column	Single column only
DEFAULT	Provides a default value when none is supplied	N/A — prevents NULL by providing a fallback	Yes — one per column	Single column only

sql – CREATE TABLE Syntax with All Constraint Types

```
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    city VARCHAR(50),
    signup_date DATE NOT NULL DEFAULT CURRENT_DATE
);
```

```
CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price DECIMAL(10,2) NOT NULL CHECK (price > 0),
```

```
stock_qty    INT                NOT NULL DEFAULT 0 CHECK (stock_qty ≥
0)
);

CREATE TABLE orders (
  order_id    SERIAL            PRIMARY KEY,
  customer_id INT              NOT NULL
                                REFERENCES customers(customer_id)
                                ON DELETE RESTRICT,
  order_date  DATE              NOT NULL DEFAULT CURRENT_DATE,
  total_amount DECIMAL(12,2)    CHECK (total_amount ≥ 0),
  status      VARCHAR(20)       NOT NULL DEFAULT 'pending'
                                CHECK (status IN
                                      ('pending','processing','shipped',
                                      'delivered','cancelled'))
);

CREATE TABLE order_items (
  order_id    INT              NOT NULL
                                REFERENCES orders(order_id)
                                ON DELETE CASCADE,
  product_id  INT              NOT NULL
                                REFERENCES products(product_id)
                                ON DELETE RESTRICT,
  quantity    INT              NOT NULL CHECK (quantity > 0),
  unit_price  DECIMAL(10,2)    NOT NULL CHECK (unit_price > 0),
  PRIMARY KEY (order_id, product_id)
);
```

PRIMARY KEY — UNIQUELY IDENTIFYING EVERY ROW



ChatGPT • AI Assistant

A two-column comparison table diagram. Left column header (gold background): 'PRIMARY KEY.' Right column header (blue background): 'UNIQUE.' Rows compare: Row 1 — 'Allows NULL': LEFT: No (red X), RIGHT: Yes (green check, one NULL per most databases). Row 2 — 'How many per table': LEFT: Exactly one, RIGHT: Multiple allowed. Row 3 — 'Enforces NOT NULL': LEFT: Yes — implicitly, RIGHT: No — must add separately. Row 4 — 'Creates index': LEFT: Yes — clustered index in most databases, RIGHT: Yes — non-clustered index. Row 5 — 'Typical use': LEFT: Row identifier — customer_id, order_id, RIGHT: Alternate identifier — email, SKU, passport. Row 6 — 'Can be composite': LEFT: Yes — (order_id, product_id), RIGHT: Yes — (first_name, last_name) though fragile. Below the table, a rule box reads: 'One table = one PRIMARY KEY, many UNIQUE constraints. PK = NOT NULL + UNIQUE combined.' Clean grid layout, gold and blue color scheme, check/X icons for boolean rows.

PRIMARY KEY vs UNIQUE — Side by Side Comparison

sql — PRIMARY KEY — Single Column, Composite, Surrogate vs Natural

```
-- Single-column surrogate PK (preferred)
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    city VARCHAR(50),
    signup_date DATE NOT NULL DEFAULT CURRENT_DATE
);

-- Natural key as PK (fragile — avoid)
CREATE TABLE customers_bad (
    email VARCHAR(150) PRIMARY KEY, -- bad: emails change
    name VARCHAR(100) NOT NULL
);

-- Composite PK on bridge/junction table
CREATE TABLE order_items (
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
    PRIMARY KEY (order_id, product_id)
);

-- MySQL syntax
CREATE TABLE customers_mysql (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL
);

-- SQL Server syntax
CREATE TABLE customers_sqlserver (
    customer_id INT IDENTITY(1,1) PRIMARY KEY,
```

```
name      NVARCHAR(100) NOT NULL,  
email     NVARCHAR(150) UNIQUE NOT NULL  
);
```

Student

Let us start with PRIMARY KEY. I know it makes a column unique — but is there more to it than that?

ChatGPT • AI Assistant

CONCEPT CARD

A **PRIMARY KEY** constraint enforces two things simultaneously on a column or set of columns:

1. **UNIQUE** — no two rows can have the same value
2. **NOT NULL** — the column cannot be empty

Every table should have exactly one primary key. A table without a primary key has no reliable way to uniquely identify a row — which makes updates, deletes, and joins fragile.

sql — Code Example

```
-- Single-column primary key  
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    name          VARCHAR(100) NOT NULL  
);  
  
-- Composite primary key (multiple columns together form the key)  
CREATE TABLE order_items (  
    order_id      INT NOT NULL,  
    product_id    INT NOT NULL,  
    quantity      INT NOT NULL,  
    PRIMARY KEY (order_id, product_id) -- neither column alone is  
unique  
);
```

Student

What is SERIAL? Is that the same as AUTO_INCREMENT in MySQL?

ChatGPT • AI Assistant

Yes — **SERIAL** is PostgreSQL's shorthand for an auto-incrementing integer:

- PostgreSQL: **SERIAL** or **GENERATED ALWAYS AS IDENTITY**
- MySQL: **INT AUTO_INCREMENT**
- SQL Server: **INT IDENTITY(1,1)**

All three do the same thing: automatically assign the next integer when a new row is inserted — you never supply the value manually.

CONCEPT CARD

Surrogate Keys vs Natural Keys:

- **Surrogate key:** a system-generated ID with no business meaning (e.g., `customer_id = 1047`). Preferred in most cases.
- **Natural key:** a real-world identifier that already exists in the data (e.g., `email`, `passport_number`, `product_sku`).

EXAMPLE CARD

Why surrogate keys are preferred:

sql – Code Example

```
-- Natural key as PK – fragile:
CREATE TABLE customers (
    email VARCHAR(150) PRIMARY KEY,    -- problem: emails change!
    name  VARCHAR(100) NOT NULL
);

-- Surrogate key – stable:
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,    -- never changes
    email        VARCHAR(150) UNIQUE NOT NULL, -- still enforced
    unique
    name        VARCHAR(100) NOT NULL
);
```

If you use `email` as the primary key and a customer changes their email, you need to cascade that change to every foreign key reference — potentially thousands of rows across many tables. A surrogate key never changes.

Composite primary key example:

sql – Code Example

```
CREATE TABLE order_items (  
    order_id    INT    NOT NULL REFERENCES orders(order_id),  
    product_id  INT    NOT NULL REFERENCES products(product_id),  
    quantity    INT    NOT NULL CHECK (quantity > 0),  
    unit_price  DECIMAL(10,2) NOT NULL,  
    PRIMARY KEY (order_id, product_id)  
    -- One order cannot contain the same product twice  
    -- order_id alone is not unique (one order has many items)  
    -- product_id alone is not unique (same product in many orders)  
    -- Together they uniquely identify each line item  
);
```

⚠ WARNING

Do not confuse PRIMARY KEY with UNIQUE. A table can have many UNIQUE constraints but only ONE PRIMARY KEY. UNIQUE allows NULL (a NULL is not considered a duplicate of another NULL in most databases). PRIMARY KEY never allows NULL.

⚠ COMMON PITFALL / MISTAKE

✗ Two primary keys on one table:

sql – Code Example

```
CREATE TABLE products (  
    product_id  INT PRIMARY KEY,  
    sku         VARCHAR(50) PRIMARY KEY    -- ERROR: only one PK  
    allowed  
);
```

✓ Correct — one PK, additional uniqueness via UNIQUE:

sql – Code Example

```
CREATE TABLE products (  
    product_id  SERIAL          PRIMARY KEY,  
    sku         VARCHAR(50) UNIQUE NOT NULL,    -- separately enforced  
    name        VARCHAR(150) NOT NULL  
);
```

 INTERVIEW CARD

Frequency: Very High

Question: What is the difference between PRIMARY KEY and UNIQUE constraint?

Expected Answer: PRIMARY KEY enforces both uniqueness and NOT NULL on a column — no two rows can have the same value and the column cannot be empty. A table can have only one PRIMARY KEY. UNIQUE enforces uniqueness but allows NULL values — and most databases allow multiple NULLs in a UNIQUE column because NULL is not considered equal to NULL for constraint checking. A table can have multiple UNIQUE constraints. PRIMARY KEY is typically used on the main row identifier; UNIQUE is used on alternate identifiers like email or SKU.

Follow-up: What is a composite primary key and when do you use one? (Answer: A composite PK uses two or more columns together as the key. Use it on junction/bridge tables like `order_items` where neither column alone is unique but the combination is.)

 PRACTICE QUIZ

1. What two constraints does PRIMARY KEY implicitly enforce?
2. Can a table have two PRIMARY KEY constraints? Can it have two UNIQUE constraints?
3. What is the difference between a surrogate key and a natural key? Which is preferred?
4. In a composite PK on (*orderid*, *productid*) — can you insert two rows with the same *orderid* but *different productid*?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

 ANSWER KEY

1. NOT NULL and UNIQUE — both applied together on the key column(s).
2. No to two PKs — a table can have exactly one PRIMARY KEY. Yes to two UNIQUE constraints — multiple allowed.
3. Surrogate key: system-generated, no business meaning, stable. Natural key: real-world attribute, may change. Surrogate keys are preferred because they never change and avoid cascade updates.
4. Yes — the composite key only requires the combination (*orderid*, *productid*) to be unique. Same *orderid* with *different productid* values is perfectly valid.

 CHECKPOINT QUESTION

Design the PRIMARY KEY strategy for a `employees` table with columns: `employeeid`, `nationalid` (government ID), email, name, department. Which column(s) should be the PK? Which should have UNIQUE? Explain your reasoning.

FOREIGN KEY AND REFERENTIAL INTEGRITY — ON DELETE OPTIONS

ChatGPT • AI Assistant

A three-panel diagram, each showing the same scenario: `customer_id` 101 in `customers` table referenced by 5 rows in `orders` table. Panel 1 (red background, label 'ON DELETE RESTRICT'): a red X icon over the DELETE arrow from `customers`. Text reads: 'DELETE blocked. Error raised. Parent row cannot be deleted while child rows exist. Safest option.' Panel 2 (orange background, label 'ON DELETE CASCADE'): the DELETE arrow from `customers` flows down with a cascade effect shown by a chain icon. All 5 order rows are struck through and labeled 'auto-deleted.' Text reads: 'Parent deleted → all child rows deleted automatically. Silent and permanent. Use carefully.' Panel 3 (blue background, label 'ON DELETE SET NULL'): the DELETE arrow from `customers` flows down to the 5 order rows. Each order row's `customer_id` column shows a transformation from '101' to 'NULL' highlighted in orange. Text reads: 'Parent deleted → FK column set to NULL in child rows. Rows still exist but are orphaned. FK column must be nullable.' A rule box at the bottom reads: 'Default is RESTRICT. Add CASCADE only where child rows have no independent meaning. SET NULL requires nullable FK column.' Clean three-panel layout, consistent iconography.

ON DELETE Options — RESTRICT vs CASCADE vs SET NULL

sql — FOREIGN KEY — All Three ON DELETE Options with Examples

```
-- ON DELETE RESTRICT: block deletion of customers with orders
CREATE TABLE orders (
  order_id      SERIAL      PRIMARY KEY,
  customer_id   INT         NOT NULL
                        REFERENCES customers(customer_id)
                        ON DELETE RESTRICT,
  order_date    DATE        NOT NULL DEFAULT CURRENT_DATE,
  total_amount  DECIMAL(12,2) CHECK (total_amount ≥ 0),
  status        VARCHAR(20) NOT NULL DEFAULT 'pending'
);

-- ON DELETE CASCADE: auto-delete order_items when order deleted
CREATE TABLE order_items (
  order_id      INT         NOT NULL
                        REFERENCES orders(order_id)
                        ON DELETE CASCADE,
  product_id    INT         NOT NULL
                        REFERENCES products(product_id)
                        ON DELETE RESTRICT,
  quantity      INT         NOT NULL CHECK (quantity > 0),
  unit_price    DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
```

```
PRIMARY KEY (order_id, product_id)
);

-- ON DELETE SET NULL: orphan reports when manager deleted
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(50),
    salary DECIMAL(10,2) CHECK (salary > 0),
    manager_id INT REFERENCES employees(employee_id)
                                ON DELETE SET NULL -- nullable: no NOT
NULL
);

-- Test RESTRICT (will error):
-- DELETE FROM customers WHERE customer_id = 101;
-- ERROR: violates foreign key constraint on table 'orders'

-- Test CASCADE (silently deletes child rows):
-- DELETE FROM orders WHERE order_id = 1001;
-- All order_items for order 1001 are deleted automatically
```

Student

Now FOREIGN KEY. I know it links two tables — but what exactly does it enforce, and what happens when you delete a parent row?

ChatGPT • AI Assistant

CONCEPT CARD

A **FOREIGN KEY** enforces **referential integrity** — it guarantees that a value in the child table's column always references a real, existing row in the parent table. You cannot insert an order for a customer that does not exist. You cannot reference a non-existent product.

sql — Code Example

```
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL
                    REFERENCES customers(customer_id)
```

```
ON DELETE RESTRICT
```

```
);
```

REFERENCES `customers(customer_id)` = *customerid in orders must exist in customers.customerid.*

Student

What happens when I try to delete a customer who has orders?

ChatGPT • AI Assistant

CONCEPT CARD

There are three options — specified with **ON DELETE** :

Option 1 — ON DELETE RESTRICT (safest)

sql — Code Example

```
customer_id INT NOT NULL REFERENCES customers(customer_id) ON DELETE RESTRICT
```

The delete is **blocked**. If you try to delete customer 101 who has 5 orders, the database throws an error. The parent row cannot be deleted while child rows reference it. This is the default behavior in most databases.

Option 2 — ON DELETE CASCADE (powerful but dangerous)

sql — Code Example

```
order_id INT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE
```

Deleting the parent row **automatically deletes all matching child rows**. Delete order 1001 → *all orderitems rows for order 1001 are deleted automatically. Appropriate for orderitems* (no order_items should exist without their parent order). Dangerous for customers → orders (you almost never want to silently delete all orders when a customer is deleted).

Option 3 — ON DELETE SET NULL (orphan with awareness)

sql — Code Example

```
manager_id INT REFERENCES employees(employee_id) ON DELETE SET NULL
```

Deleting the parent row sets the FK column in all child rows to **NULL**. Delete an employee who is someone's manager → their direct reports' manager_id becomes NULL. They are not deleted — just left without a manager. Only works if the FK column is nullable (no NOT NULL constraint).

EXAMPLE CARD

Example 1 — ON DELETE RESTRICT: Protect customers from accidental deletion

sql – Code Example

```
CREATE TABLE orders (
    order_id      SERIAL          PRIMARY KEY,
    customer_id   INT             NOT NULL
                                REFERENCES customers(customer_id)
                                ON DELETE RESTRICT,
    order_date    DATE            NOT NULL DEFAULT CURRENT_DATE,
    total_amount  DECIMAL(12,2) CHECK (total_amount ≥ 0),
    status        VARCHAR(20)     NOT NULL DEFAULT 'pending'
);

-- Try to delete a customer with orders:
DELETE FROM customers WHERE customer_id = 101;
-- ERROR: update or delete on table 'customers' violates foreign key
-- constraint on table 'orders'
```

Example 2 — ON DELETE CASCADE: order_items are meaningless without their order

sql – Code Example

```
CREATE TABLE order_items (
    order_id      INT             NOT NULL
                                REFERENCES orders(order_id)
                                ON DELETE CASCADE,
    product_id    INT             NOT NULL
                                REFERENCES products(product_id)
                                ON DELETE RESTRICT,
    quantity      INT             NOT NULL CHECK (quantity > 0),
    unit_price    DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (order_id, product_id)
);
```

```
-- Delete order 1001:
DELETE FROM orders WHERE order_id = 1001;
-- Automatically deletes all order_items WHERE order_id = 1001
-- No error. Silent cascade.
```

Example 3 — ON DELETE SET NULL: Manager hierarchy

sql – Code Example

```
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(50),
    manager_id INT REFERENCES employees(employee_id)
                        ON DELETE SET NULL
    -- manager_id is nullable – no NOT NULL constraint
);

-- Delete a manager:
DELETE FROM employees WHERE employee_id = 50; -- Carol the manager
-- Carol's direct reports now have manager_id = NULL
-- They still exist – just without a manager assigned
```

⚠ WARNING

ON DELETE CASCADE in production is dangerous. If you accidentally delete a parent row — or if a bug triggers a DELETE — the cascade silently removes potentially thousands of child rows across multiple tables. Always double-check that CASCADE is intentional. Use RESTRICT as the default and add CASCADE only where genuinely needed.

❗ COMMON PITFALL / MISTAKE

✗ Adding NOT NULL to a SET NULL foreign key column:

sql – Code Example

```
manager_id INT NOT NULL REFERENCES employees(employee_id) ON DELETE
SET NULL
-- ERROR: cannot use ON DELETE SET NULL on a NOT NULL column
-- If the parent is deleted, SET NULL cannot fulfill NOT NULL
```

✅ Correct — SET NULL requires a nullable column:

sql – Code Example

```
manager_id INT REFERENCES employees(employee_id) ON DELETE SET NULL
-- No NOT NULL -- allows the column to become NULL
```

INTERVIEW CARD

Frequency: Very High

Question: What are the three ON DELETE options for a FOREIGN KEY? When do you use each?

Expected Answer: ON DELETE RESTRICT (default) blocks the delete if child rows exist — safest, prevents accidental data loss. ON DELETE CASCADE automatically deletes all child rows when the parent is deleted — use only where child rows have no meaning without the parent, such as order_items without their order. ON DELETE SET NULL sets the FK column to NULL in child rows when the parent is deleted — use for optional relationships like manager assignments. Important: ON DELETE SET NULL requires the FK column to be nullable — adding NOT NULL to a SET NULL column is a constraint conflict error.

Follow-up: What is the difference between ON DELETE RESTRICT and ON DELETE NO ACTION? (Answer: In most databases they behave identically. RESTRICT is checked immediately; NO ACTION is deferred to end of transaction. The practical difference only matters in deferred constraint transactions.)

PRACTICE QUIZ

1. What does ON DELETE RESTRICT do when you try to delete a parent row that has child rows?
2. Which ON DELETE option should you use for order_items when the parent order is deleted?
3. Why does ON DELETE SET NULL require the FK column to be nullable?
4. You delete a product that has been ordered 500 times via order_items. The FK on order_items.product_id uses ON DELETE RESTRICT. What happens?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. The DELETE is blocked with an error — child rows must be deleted or reassigned first.

2. ON DELETE CASCADE — order_items rows have no meaning without their parent order.
3. Because when the parent is deleted, the database sets the FK column to NULL. If the column has NOT NULL, the database cannot fulfill the constraint — it is a logical conflict.
4. The delete is blocked — error message states the FK constraint is violated. The product row remains in the database. You must delete or reassign the 500 order_items rows first.

🔗 CHECKPOINT QUESTION

You are designing a `reviews` table where customers leave reviews on products: `reviews(review_id, customer_id, product_id, rating, comment, created_at)`. Define the correct ON DELETE option for each FK and explain your reasoning. What happens if a customer account is deleted? What happens if a product is discontinued?

UNIQUE, CHECK, AND NOT NULL CONSTRAINTS

ChatGPT • AI Assistant

A three-row decision table diagram showing how a CHECK constraint evaluates different input values. Column headers: 'Input Value', 'CHECK (price > 0) Evaluation', 'Result', 'Row Accepted?'. Row 1 (green background): Input: 500.00. Evaluation: 500 > 0 = TRUE. Result: TRUE. Accepted: Yes (green check). Row 2 (red background): Input: -10.00. Evaluation: -10 > 0 = FALSE. Result: FALSE. Accepted: No (red X) — CHECK violation error. Row 3 (orange background): Input: NULL. Evaluation: NULL > 0 = NULL. Result: NULL (not FALSE). Accepted: Yes (orange warning check) — 'NULL slips through CHECK!' Warning annotation reads: 'NULL is not FALSE — it passes the CHECK. Add NOT NULL separately to block NULLs.' A solution box reads: 'price DECIMAL NOT NULL CHECK (price > 0) — two constraints working together.' Clean table layout, color-coded rows, warning icon on NULL row.

CHECK Constraint NULL Behaviour — Why NOT NULL + CHECK Are Needed Together

sql — UNIQUE, CHECK, NOT NULL — Inline, Table-Level, and Named Constraints

```
-- NOT NULL: required fields
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    city VARCHAR(50), -- nullable: optional field
    signup_date DATE NOT NULL DEFAULT CURRENT_DATE
);

-- UNIQUE: single and composite
CREATE TABLE products (
```

```

product_id SERIAL PRIMARY KEY,
sku VARCHAR(50) UNIQUE NOT NULL,
name VARCHAR(150) NOT NULL,
category VARCHAR(50) NOT NULL,
price DECIMAL(10,2) NOT NULL CHECK (price > 0),
stock_qty INT NOT NULL DEFAULT 0 CHECK (stock_qty ≥ 0)
);

-- Table-level CHECK spanning two columns
CREATE TABLE promotions (
    promo_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    discount DECIMAL(5,2) NOT NULL CHECK (discount > 0 AND discount
≤ 100),
    CHECK (end_date > start_date)
);

-- Named constraints (production best practice)
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL
        REFERENCES customers(customer_id)
        ON DELETE RESTRICT,
    total_amount DECIMAL(12,2) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    order_date DATE NOT NULL DEFAULT CURRENT_DATE,
    delivery_date DATE,

    CONSTRAINT chk_amount_nonneg CHECK (total_amount ≥ 0),
    CONSTRAINT chk_status_valid CHECK (status IN
        ('pending','processing','shipped',
        'delivered','cancelled')),
    CONSTRAINT chk_delivery_after CHECK (
        delivery_date IS NULL
        OR delivery_date ≥ order_date)
);

-- Drop a named constraint
ALTER TABLE orders DROP CONSTRAINT chk_status_valid;

```

Let us cover the remaining three constraints — UNIQUE, CHECK, and NOT NULL.

ChatGPT • AI Assistant

CONCEPT CARD

NOT NULL is the simplest — it prevents a column from ever containing a NULL value. A column with no NOT NULL constraint is nullable by default.

UNIQUE ensures all values in a column (or combination of columns) are distinct. Unlike PRIMARY KEY, UNIQUE allows NULL — and in most databases, multiple NULLs are allowed (because NULL ≠ NULL in SQL).

CHECK enforces a custom boolean condition on a column's value. If the condition evaluates to FALSE, the insert or update is rejected. If it evaluates to NULL (because the column is NULL), the CHECK passes — NULL is not FALSE.

EXAMPLE CARD

Example 1 — NOT NULL: Required fields

sql — Code Example

```
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,           -- name is required  
    email VARCHAR(150) NOT NULL UNIQUE,   -- required and unique  
    city VARCHAR(50),                     -- nullable: city is  
optional  
    signup_date DATE NOT NULL DEFAULT CURRENT_DATE  
);  
  
-- Try to insert without a name:  
INSERT INTO customers (email) VALUES ('test@example.com');  
-- ERROR: null value in column 'name' violates not-null constraint
```

Example 2 — UNIQUE: Enforce unique email and unique SKU

sql — Code Example

```
CREATE TABLE products (  
    product_id SERIAL PRIMARY KEY,  
    sku VARCHAR(50) UNIQUE NOT NULL,      -- no two products  
share a SKU  
    name VARCHAR(150) NOT NULL,
```

```
category    VARCHAR(50)    NOT NULL,
price       DECIMAL(10,2) NOT NULL CHECK (price > 0)
);

-- Composite UNIQUE: no two products in the same category can share a
name
CREATE TABLE products (
    product_id SERIAL          PRIMARY KEY,
    name          VARCHAR(150) NOT NULL,
    category      VARCHAR(50)  NOT NULL,
    price         DECIMAL(10,2) NOT NULL,
    UNIQUE (category, name)    -- composite unique constraint
);
```

Example 3 — CHECK: Business rule enforcement

sql – Code Example

```
CREATE TABLE products (
    product_id SERIAL          PRIMARY KEY,
    name          VARCHAR(150) NOT NULL,
    price         DECIMAL(10,2) NOT NULL CHECK (price > 0),
    stock_qty     INT           NOT NULL CHECK (stock_qty ≥ 0),
    discount      DECIMAL(5,2)          CHECK (discount BETWEEN 0 AND
100)
);

CREATE TABLE orders (
    order_id      SERIAL          PRIMARY KEY,
    customer_id   INT             NOT NULL REFERENCES
customers(customer_id),
    status        VARCHAR(20) NOT NULL DEFAULT 'pending'
                                CHECK (status IN
                                    ('pending','processing','shipped',
                                    'delivered','cancelled')),
    order_date    DATE            NOT NULL DEFAULT CURRENT_DATE,
    delivery_date DATE,
    CHECK (delivery_date IS NULL OR delivery_date ≥ order_date)
    -- table-level CHECK: delivery cannot be before order date
);
```

Student

Can I add CHECK across two columns — like making sure *enddate is after startdate*?

ChatGPT • AI Assistant

Yes — that requires a **table-level CHECK constraint**, defined after all the column definitions:

sql – Code Example

```
CREATE TABLE promotions (  
    promo_id    SERIAL          PRIMARY KEY,  
    name        VARCHAR(100) NOT NULL,  
    start_date   DATE           NOT NULL,  
    end_date     DATE           NOT NULL,  
    discount     DECIMAL(5,2) NOT NULL CHECK (discount > 0 AND discount  
≤ 100),  
    CHECK (end_date > start_date)  -- table-level: spans two columns  
);  
  
-- Test:  
INSERT INTO promotions (name, start_date, end_date, discount)  
VALUES ('Summer Sale', '2024-08-01', '2024-07-01', 20);  
-- ERROR: new row violates check constraint 'promotions_check'
```

Example 4 — Named constraints (best practice for production)

sql – Code Example

```
CREATE TABLE products (  
    product_id  SERIAL          PRIMARY KEY,  
    price       DECIMAL(10,2) NOT NULL,  
    stock_qty   INT             NOT NULL,  
    status      VARCHAR(20)     NOT NULL,  
  
    CONSTRAINT chk_price_positive    CHECK (price > 0),  
    CONSTRAINT chk_stock_nonnegative CHECK (stock_qty ≥ 0),  
    CONSTRAINT chk_status_valid      CHECK (status IN  
  
('active','inactive','discontinued'))  
);
```

Naming constraints makes error messages readable and lets you drop/modify specific constraints by name later:

sql – Code Example

```
ALTER TABLE products DROP CONSTRAINT chk_price_positive;
```

WARNING

A NULL value passes a CHECK constraint — because `NULL > 0` evaluates to NULL (not FALSE), and NULL is not FALSE. If you want to reject NULL AND enforce a check, you need both NOT NULL and CHECK together:

sql – Code Example

```
price DECIMAL(10,2) NOT NULL CHECK (price > 0)
-- NOT NULL rejects NULL. CHECK rejects zero and negative values.
```

INTERVIEW CARD

Frequency: High

Question: What happens if a column value is NULL — does it pass a CHECK constraint?

Expected Answer: Yes — a NULL value passes a CHECK constraint because the check expression evaluates to NULL (not FALSE), and SQL only rejects rows where the CHECK evaluates to FALSE. If you want to prevent NULL and also enforce a range, you need both NOT NULL and CHECK on the same column. For example, `price DECIMAL NOT NULL CHECK (price > 0)` — NOT NULL rejects NULL, and CHECK rejects zero or negative values.

Follow-up: What is a table-level CHECK constraint and when do you need one? (Answer: A table-level CHECK is defined after all column definitions and can reference multiple columns. You need it when the constraint logic spans more than one column — e.g., `CHECK (enddate > startdate)` or `CHECK (discount < price)`.)

PRACTICE QUIZ

1. Does UNIQUE allow NULL values? Can a column have multiple NULL values with a UNIQUE constraint?
2. What is the difference between a column-level and a table-level CHECK constraint?

3. Why is naming your constraints (CONSTRAINT chk_name CHECK ...) a best practice?
4. Predict: does this insert succeed? `INSERT INTO products (name, price) VALUES ('Widget', -5);` if the table has `price DECIMAL NOT NULL CHECK (price > 0)`.

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. Yes, UNIQUE allows NULL. In most databases (PostgreSQL, SQL Server, Oracle) multiple NULLs are allowed in a UNIQUE column because NULL is not considered equal to NULL.
2. Column-level CHECK applies to a single column and is written inline. Table-level CHECK is written after all column definitions and can span multiple columns.
3. Named constraints produce readable error messages and can be dropped or modified by name with ALTER TABLE DROP CONSTRAINT constraint_name.
4. No — the insert fails with a CHECK constraint violation because `-5 > 0` evaluates to FALSE.

🔗 **CHECKPOINT QUESTION**

Design the full CREATE TABLE statement for a `coupons` table with columns: `couponid` (PK auto-increment), `code` (must be unique and not null), `discountpercent` (must be between 1 and 100), `validfrom` (date, not null), `validuntil` (date, not null), `isactive` (boolean, default true). Add a table-level CHECK ensuring `validuntil` is after `valid_from`. Name all CHECK constraints.

FULL SCHEMA WALKTHROUGH — DESIGNING THE ORDER MANAGEMENT SYSTEM

ChatGPT • AI Assistant

A four-table entity-relationship diagram identical in structure to the section overview figure but with detailed constraint annotations on every column. Each column row has a right-side annotation badge showing which constraints apply and why. customers table annotations: customer_id — 'SERIAL PK: auto-increment, stable FK target.' email — 'UNIQUE NOT NULL: required alternate identifier.' city — 'nullable: optional field.' signup_date — 'NOT NULL DEFAULT: always recorded.' products table annotations: price — 'NOT NULL CHECK >0: required and must be positive.' stock_qty — 'DEFAULT 0 CHECK >=0: starts at zero, cannot go negative.' orders table annotations: customer_id FK — 'ON DELETE RESTRICT: protect order history.' status — 'CHECK IN list: closed enumeration, no freeform strings.' delivery_date — 'nullable CHECK >= order_date: only set on delivery.' order_items table annotations: order_id FK — 'ON DELETE CASCADE: items meaningless without order.' product_id FK — 'ON DELETE RESTRICT: preserve order history.' PRIMARY KEY badge on (order_id, product_id) — 'COMPOSITE PK: one product per order line.' All annotations use color-coded badges: green for PK, orange for FK, red for NOT NULL, blue for UNIQUE, purple for CHECK, gray for DEFAULT.

Full Order Management Schema — Constraint Decision Map

sql – Complete Order Management Schema – All Four Tables Fully Constrained

```
-- =====
-- COMPLETE ORDER MANAGEMENT SCHEMA – FULLY CONSTRAINED
-- =====

CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    city VARCHAR(50),
    signup_date DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock_qty INT NOT NULL DEFAULT 0,
    CONSTRAINT chk_price_positive CHECK (price > 0),
    CONSTRAINT chk_stock_nonnegative CHECK (stock_qty ≥ 0)
);

CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL
        REFERENCES customers(customer_id)
        ON DELETE RESTRICT,
    order_date DATE NOT NULL DEFAULT CURRENT_DATE,
    total_amount DECIMAL(12,2),
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    delivery_date DATE,
    CONSTRAINT chk_total_nonneg CHECK (total_amount IS NULL OR
total_amount ≥ 0),
```

```

CONSTRAINT chk_status_values CHECK (status IN
    ('pending','processing','shipped',
    'delivered','cancelled')),
CONSTRAINT chk_delivery_order CHECK (
    delivery_date IS NULL
    OR delivery_date ≥ order_date)
);

CREATE TABLE order_items (
    order_id INT NOT NULL
        REFERENCES orders(order_id)
        ON DELETE CASCADE,
    product_id INT NOT NULL
        REFERENCES products(product_id)
        ON DELETE RESTRICT,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (order_id, product_id),
    CONSTRAINT chk_qty_positive CHECK (quantity > 0),
    CONSTRAINT chk_unit_price_positive CHECK (unit_price > 0)
);

-- =====
-- ALTER TABLE: add constraints to existing tables
-- =====

ALTER TABLE products
    ADD CONSTRAINT chk_price_positive CHECK (price > 0);

ALTER TABLE customers
    ADD CONSTRAINT uq_customer_email UNIQUE (email);

ALTER TABLE orders
    ADD CONSTRAINT fk_orders_customer
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    ON DELETE RESTRICT;

```

Student

Can we now put it all together — design the full order management schema with every constraint explained?

CONCEPT CARD

Here is the complete, production-quality schema for the order management system. Every constraint is present and every decision is explained.

EXAMPLE CARD**Step 1 — customers table**

sql – Code Example

```
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    -- Surrogate PK: auto-increments, never changes, stable FK target  
  
    name VARCHAR(100) NOT NULL,  
    -- NOT NULL: every customer must have a name  
  
    email VARCHAR(150) NOT NULL UNIQUE,  
    -- NOT NULL + UNIQUE: required and no two customers share an email  
  
    city VARCHAR(50),  
    -- Nullable: city is optional – customer may not provide it  
  
    signup_date DATE NOT NULL DEFAULT CURRENT_DATE  
    -- NOT NULL + DEFAULT: always recorded, defaults to today if not  
    supplied  
);
```

Step 2 — products table

sql – Code Example

```
CREATE TABLE products (  
    product_id SERIAL PRIMARY KEY,  
    -- Surrogate PK  
  
    name VARCHAR(150) NOT NULL,  
    -- Required: every product needs a name  
  
    category VARCHAR(50) NOT NULL,  
    -- Required: products must belong to a category  
  
    price DECIMAL(10,2) NOT NULL,  
    -- NOT NULL: price is required
```

```

stock_qty    INT                NOT NULL DEFAULT 0,
-- Default 0: new products start with zero stock

CONSTRAINT chk_price_positive    CHECK (price > 0),
-- Price must be positive: zero-priced products make no sense

CONSTRAINT chk_stock_nonnegative CHECK (stock_qty ≥ 0)
-- Stock cannot go negative: a logical impossibility
);

```

Step 3 — orders table

sql – Code Example

```

CREATE TABLE orders (
    order_id    SERIAL            PRIMARY KEY,
    -- Surrogate PK

    customer_id INT              NOT NULL
                                REFERENCES customers(customer_id)
                                ON DELETE RESTRICT,
    -- FK to customers. RESTRICT: block customer deletion if orders
    exist.
    -- We never want to silently delete a customer's order history.

    order_date  DATE              NOT NULL DEFAULT CURRENT_DATE,
    -- Required and defaults to today

    total_amount DECIMAL(12,2),
    -- Nullable: may be computed after items are added

    status      VARCHAR(20)       NOT NULL DEFAULT 'pending',
    -- Defaults to pending on creation

    delivery_date DATE,
    -- Nullable: set only when order is delivered

    CONSTRAINT chk_total_nonneg    CHECK (total_amount IS NULL OR
total_amount ≥ 0),
    -- Allow NULL (pre-computation) but reject negative values

    CONSTRAINT chk_status_values   CHECK (status IN
('pending','processing','shipped',
                                'delivered','cancelled')),
    -- Closed list: no freeform status strings

```

```
CONSTRAINT chk_delivery_order CHECK (  
                                delivery_date IS NULL  
                                OR delivery_date ≥ order_date)  
-- Delivery cannot precede the order date  
);
```

Step 4 — order_items table

sql – Code Example

```
CREATE TABLE order_items (  
    order_id      INT                NOT NULL  
                                REFERENCES orders(order_id)  
                                ON DELETE CASCADE,  
    -- CASCADE: if the parent order is deleted, its items must also be  
    -- deleted.  
    -- Line items have no meaning without their order.  
  
    product_id    INT                NOT NULL  
                                REFERENCES products(product_id)  
                                ON DELETE RESTRICT,  
    -- RESTRICT: do not allow a product to be deleted if it has been  
    -- ordered.  
    -- Historical order data must remain intact.  
  
    quantity      INT                NOT NULL,  
    unit_price     DECIMAL(10,2)     NOT NULL,  
  
    PRIMARY KEY (order_id, product_id),  
    -- Composite PK: each product appears at most once per order  
  
    CONSTRAINT chk_qty_positive      CHECK (quantity > 0),  
    CONSTRAINT chk_unit_price_positive CHECK (unit_price > 0)  
    -- Both must be positive: zero quantity or price is logically  
    invalid  
);
```

Student

How do I add a constraint to an existing table after it is already created?

Use **ALTER TABLE** :

sql – Code Example

```
-- Add a CHECK constraint to an existing table
ALTER TABLE products
    ADD CONSTRAINT chk_price_positive CHECK (price > 0);

-- Add a NOT NULL constraint
ALTER TABLE customers
    ALTER COLUMN name SET NOT NULL;

-- Add a UNIQUE constraint
ALTER TABLE customers
    ADD CONSTRAINT uq_customer_email UNIQUE (email);

-- Add a FOREIGN KEY constraint
ALTER TABLE orders
    ADD CONSTRAINT fk_orders_customer
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    ON DELETE RESTRICT;

-- Drop a named constraint
ALTER TABLE orders
    DROP CONSTRAINT chk_status_values;
```

WARNING

Adding a constraint to an existing table with data will **validate all existing rows**. If any row violates the new constraint, the ALTER TABLE fails. You must clean the data first:

sql – Code Example

```
-- First check for violations:
SELECT * FROM products WHERE price ≤ 0;

-- Fix violations:
UPDATE products SET price = 0.01 WHERE price ≤ 0;

-- Then add the constraint:
ALTER TABLE products ADD CONSTRAINT chk_price_positive CHECK (price > 0);
```

INTERVIEW CARD

Frequency: High

Question: Walk me through designing a schema with appropriate constraints. What decisions do you make for each table?

Expected Answer: For each table: choose a surrogate PK (SERIAL/AUTOINCREMENT) *unless a natural key is truly stable and unique*. Add NOT NULL to every required column. Add UNIQUE for alternate identifiers like email or SKU. Add CHECK constraints for business rules — positive prices, non-negative stock, valid status values, date ordering. Choose ON DELETE options for FK columns carefully: RESTRICT for most relationships to prevent accidental data loss; CASCADE only where child rows have no independent meaning (like orderitems); SET NULL for optional relationships. Name all constraints so error messages are readable and constraints can be dropped by name. Add DEFAULT values for columns that have a sensible starting value.

Follow-up: What happens when you run ALTER TABLE ADD CONSTRAINT on a table that already has data violating the constraint? (Answer: The command fails with a constraint violation error — you must fix all violating rows first, then add the constraint.)

🔍 PRACTICE QUIZ

1. Why do you use ON DELETE RESTRICT for orders.customerid but ON DELETE CASCADE for orderitems.order_id?
2. Why is total_amount in orders nullable but has a CHECK that rejects negative values?
3. What ALTER TABLE command adds a CHECK constraint to an existing table?
4. You want to add CHECK (price > 0) to a products table that already has 3 rows with price = 0. What happens and how do you fix it?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. RESTRICT on orders: we never want to silently delete order history when a customer is removed. CASCADE on order_items: line items have no meaning without their parent order — deleting an order should clean up its items automatically.
2. total_amount is nullable because it may be computed after items are added (the order is created before items are inserted). The CHECK allows NULL (pre-computation) but rejects negative values once a value is present.

3. `ALTER TABLE table_name ADD CONSTRAINT constraint_name CHECK (condition);`
4. The ALTER TABLE fails with a constraint violation error for the 3 rows with price = 0. Fix: `UPDATE products SET price = 0.01 WHERE price ≤ 0;` then re-run the ALTER TABLE.

🔗 CHECKPOINT QUESTION

Before the practice problems: without running it, identify every missing or incorrect constraint in this under-constrained schema and write the corrected full CREATE TABLE statements:

sql – Code Example

```
CREATE TABLE hotels (  
    hotel_id    INT,  
    name        VARCHAR(100),  
    city        VARCHAR(50),  
    star_rating INT  
);  
  
CREATE TABLE rooms (  
    room_id     INT,  
    hotel_id    INT,  
    room_number VARCHAR(10),  
    price       DECIMAL(10,2),  
    capacity    INT  
);  
  
CREATE TABLE reservations (  
    reservation_id INT,  
    room_id        INT,  
    guest_name     VARCHAR(100),  
    check_in       DATE,  
    check_out      DATE  
);
```

PRACTICE PROBLEMS — CONSTRAINED SCHEMA DESIGN

ChatGPT • AI Assistant

A decision flowchart for schema design with five parallel decision branches, one per constraint type. Branch 1 (gold): 'Does this column uniquely identify each row?' YES → 'Add PRIMARY KEY. Use SERIAL for auto-increment. Exactly one per table.' Branch 2 (orange): 'Does this column reference a row in another table?' YES → 'Add FOREIGN KEY REFERENCES. Choose ON DELETE: RESTRICT (default/safe), CASCADE (child depends on parent), SET NULL (optional relationship).' Branch 3 (blue): 'Must all non-NULL values in this column be distinct?' YES → 'Add UNIQUE. Allows NULLs. Use for email, SKU, alternate identifiers.' Branch 4 (red): 'Is this column always required — should it never be empty?' YES → 'Add NOT NULL. Combine with DEFAULT if a sensible fallback exists.' Branch 5 (purple): 'Must the column value satisfy a business rule (range, list, comparison)?' YES → 'Add CHECK constraint. Name it. Use table-level CHECK for multi-column rules. Remember: NULL passes CHECK — add NOT NULL too if needed.' All five branches reconnect at the bottom to a green box: 'Complete column definition — column_name TYPE [NOT NULL] [DEFAULT val] [UNIQUE] [CHECK (condition)] [REFERENCES table(col) ON DELETE action].' Clean parallel flowchart layout, color-coded branches.

Schema Design Decision Flowchart — Choosing Constraints for Each Column

sql – Practice Problems – Under-Constrained Fix and Hotel Booking Schema

```
-- =====
-- PROBLEM 1 SOLUTION: Fully constrained employees/projects
-- =====

CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    department VARCHAR(50) NOT NULL,
    salary DECIMAL(10,2) NOT NULL,
    manager_id INT REFERENCES employees(employee_id)
    ON DELETE SET NULL,
    hire_date DATE NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT chk_salary_positive CHECK (salary > 0)
);

CREATE TABLE projects (
    project_id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL UNIQUE,
    start_date DATE NOT NULL,
    end_date DATE,
    budget DECIMAL(15,2) NOT NULL,
    CONSTRAINT chk_budget_positive CHECK (budget > 0),
    CONSTRAINT chk_dates_order CHECK (end_date IS NULL OR end_date
    ≥ start_date)
);

CREATE TABLE project_assignments (
    employee_id INT NOT NULL REFERENCES
employees(employee_id) ON DELETE RESTRICT,
    project_id INT NOT NULL REFERENCES projects(project_id)
ON DELETE RESTRICT,
    role VARCHAR(50) NOT NULL,
    hours_week DECIMAL(4,1) NOT NULL,
```

```
PRIMARY KEY (employee_id, project_id),
CONSTRAINT chk_hours_valid CHECK (hours_week > 0 AND hours_week ≤
60)
);

-- =====
-- PROBLEM 2 SOLUTION: Hotel booking system
-- =====

CREATE TABLE hotels (
    hotel_id    SERIAL          PRIMARY KEY,
    name        VARCHAR(200)    NOT NULL,
    city        VARCHAR(100)    NOT NULL,
    star_rating INT              NOT NULL,
    CONSTRAINT chk_star_rating CHECK (star_rating BETWEEN 1 AND 5)
);

CREATE TABLE rooms (
    room_id     SERIAL          PRIMARY KEY,
    hotel_id    INT              NOT NULL REFERENCES hotels(hotel_id) ON
DELETE CASCADE,
    room_number VARCHAR(10)     NOT NULL,
    room_type   VARCHAR(20)     NOT NULL,
    nightly_rate DECIMAL(10,2) NOT NULL,
    capacity    INT              NOT NULL,
    UNIQUE (hotel_id, room_number),
    CONSTRAINT chk_room_type    CHECK (room_type IN
('single','double','suite','deluxe')),
    CONSTRAINT chk_rate_positive CHECK (nightly_rate > 0),
    CONSTRAINT chk_capacity_min  CHECK (capacity ≥ 1)
);

CREATE TABLE guests (
    guest_id SERIAL          PRIMARY KEY,
    name     VARCHAR(100)    NOT NULL,
    email    VARCHAR(150)    NOT NULL UNIQUE,
    phone    VARCHAR(20)
);

CREATE TABLE reservations (
    reservation_id SERIAL          PRIMARY KEY,
    room_id        INT              NOT NULL REFERENCES rooms(room_id)
ON DELETE RESTRICT,
    guest_id       INT              NOT NULL REFERENCES guests(guest_id)
ON DELETE RESTRICT,
    check_in       DATE              NOT NULL,
    check_out      DATE              NOT NULL,
```

```

total_cost      DECIMAL(12,2),
status          VARCHAR(20)  NOT NULL DEFAULT 'confirmed',
CONSTRAINT chk_dates_valid CHECK (check_out > check_in),
CONSTRAINT chk_cost_nonneg CHECK (total_cost IS NULL OR total_cost
≥ 0),
CONSTRAINT chk_res_status  CHECK (status IN

('confirmed','checked_in','checked_out','cancelled'))
);

```

Student

The prompt said two practice problems for this section. Can we do those?

ChatGPT • AI Assistant

CONCEPT CARD

The two practice problems for SQL 1.6 are:

1. **Fix an under-constrained schema** — add all missing constraints to a given schema.
2. **Design from scratch** — design a fully constrained schema for a hotel booking system.

Both are real interview tasks. Schema design questions are common in data engineering and backend engineering interviews.

Practice Problem 1 — Add all missing constraints to this under-constrained schema:

Given schema (no constraints):

sql — Code Example

```

CREATE TABLE employees (
  employee_id INT,
  name        VARCHAR(100),
  email       VARCHAR(150),
  department  VARCHAR(50),
  salary      DECIMAL(10,2),
  manager_id  INT,
  hire_date   DATE

```

```
);

CREATE TABLE projects (
    project_id    INT,
    name          VARCHAR(200),
    start_date    DATE,
    end_date      DATE,
    budget        DECIMAL(15,2)
);

CREATE TABLE project_assignments (
    employee_id   INT,
    project_id    INT,
    role          VARCHAR(50),
    hours_week    DECIMAL(4,1)
);
```

Solution — Fully constrained version:

sql – Code Example

```
CREATE TABLE employees (
    employee_id   SERIAL          PRIMARY KEY,
    -- Surrogate PK: auto-increment stable identifier

    name          VARCHAR(100)    NOT NULL,
    -- Required: cannot have a nameless employee

    email         VARCHAR(150)    NOT NULL UNIQUE,
    -- Required and unique: each employee has exactly one email

    department    VARCHAR(50)     NOT NULL,
    -- Required: every employee belongs to a department

    salary        DECIMAL(10,2)   NOT NULL,
    -- Required: salary must be recorded

    manager_id    INT             REFERENCES employees(employee_id)
                                ON DELETE SET NULL,
    -- Self-referencing FK: nullable (CEO has no manager).
    -- SET NULL: if a manager is deleted, reports are not deleted
    -- they just have no manager assigned (manager_id becomes NULL)

    hire_date     DATE            NOT NULL DEFAULT CURRENT_DATE,
    -- Required: defaults to today if not supplied
```

```
CONSTRAINT chk_salary_positive CHECK (salary > 0)
-- Salary must be positive: zero or negative makes no sense
);

CREATE TABLE projects (
    project_id SERIAL PRIMARY KEY,

    name VARCHAR(200) NOT NULL UNIQUE,
    -- Required and unique: project names must be distinct

    start_date DATE NOT NULL,
    end_date DATE,
    -- end_date nullable: project may be ongoing

    budget DECIMAL(15,2) NOT NULL,

    CONSTRAINT chk_budget_positive CHECK (budget > 0),
    CONSTRAINT chk_dates_order CHECK (
        end_date IS NULL
        OR end_date ≥ start_date)
    -- End date must be on or after start date (or NULL if ongoing)
);

CREATE TABLE project_assignments (
    employee_id INT NOT NULL
        REFERENCES employees(employee_id)
        ON DELETE RESTRICT,
    -- RESTRICT: do not delete employee if they have project history

    project_id INT NOT NULL
        REFERENCES projects(project_id)
        ON DELETE RESTRICT,
    -- RESTRICT: do not delete project if it has assigned employees

    role VARCHAR(50) NOT NULL,
    -- Required: assignment must specify a role

    hours_week DECIMAL(4,1) NOT NULL,
    -- Required: hours per week must be recorded

    PRIMARY KEY (employee_id, project_id),
    -- Composite PK: one employee assigned once per project

    CONSTRAINT chk_hours_valid CHECK (hours_week > 0 AND hours_week ≤
60)
```

```
-- Hours must be positive and capped at 60 per week
);
```

Practice Problem 2 — Design from scratch: Hotel booking system

Requirements:

- Hotels have rooms. Rooms have a type (single, double, suite) and a nightly rate.
- Guests make reservations for a specific room.
- Check-in date must be before check-out date.
- Nightly rate must be positive. Capacity must be at least 1.
- A guest's email must be unique. Name and email are required.
- Deleting a hotel should delete its rooms. Deleting a room should block if active reservations exist.

Solution — Hotel booking schema:

sql — Code Example

```
CREATE TABLE hotels (
    hotel_id      SERIAL          PRIMARY KEY,
    name          VARCHAR(200)    NOT NULL,
    city          VARCHAR(100)    NOT NULL,
    star_rating   INT             NOT NULL,
    CONSTRAINT chk_star_rating CHECK (star_rating BETWEEN 1 AND 5)
);

CREATE TABLE rooms (
    room_id      SERIAL          PRIMARY KEY,
    hotel_id     INT             NOT NULL
                                REFERENCES hotels(hotel_id)
                                ON DELETE CASCADE,
    -- CASCADE: deleting a hotel removes all its rooms

    room_number  VARCHAR(10)     NOT NULL,
    room_type    VARCHAR(20)     NOT NULL,
    nightly_rate DECIMAL(10,2)   NOT NULL,
    capacity     INT             NOT NULL,

    UNIQUE (hotel_id, room_number),
    -- Composite UNIQUE: room number must be unique within each hotel

    CONSTRAINT chk_room_type CHECK (room_type IN

('single','double','suite','deluxe')),
```

```

CONSTRAINT chk_rate_positive CHECK (nightly_rate > 0),
CONSTRAINT chk_capacity_min CHECK (capacity ≥ 1)
);

CREATE TABLE guests (
    guest_id      SERIAL          PRIMARY KEY,
    name          VARCHAR(100)    NOT NULL,
    email         VARCHAR(150)    NOT NULL UNIQUE,
    phone         VARCHAR(20)
    -- phone nullable: optional contact field
);

CREATE TABLE reservations (
    reservation_id SERIAL          PRIMARY KEY,
    room_id        INT             NOT NULL
                                REFERENCES rooms(room_id)
                                ON DELETE RESTRICT,
    -- RESTRICT: do not delete a room that has reservations

    guest_id       INT             NOT NULL
                                REFERENCES guests(guest_id)
                                ON DELETE RESTRICT,
    -- RESTRICT: do not delete a guest who has reservations

    check_in       DATE            NOT NULL,
    check_out      DATE            NOT NULL,
    total_cost     DECIMAL(12,2),
    -- Nullable: computed from nightly_rate x nights

    status         VARCHAR(20)    NOT NULL DEFAULT 'confirmed',

    CONSTRAINT chk_dates_valid CHECK (check_out > check_in),
    -- check_out must be strictly after check_in

    CONSTRAINT chk_cost_nonneg CHECK (total_cost IS NULL OR
total_cost ≥ 0),
    CONSTRAINT chk_res_status CHECK (status IN
                                ('confirmed','checked_in',
                                'checked_out','cancelled'))
);

```

SUMMARY CARD**SQL 1.6 Key Takeaways:**

- **PRIMARY KEY** = NOT NULL + UNIQUE. One per table. Use surrogate keys (SERIAL/AUTO_INCREMENT).

- **Composite PK** — use on junction tables where no single column alone is unique.
- **FOREIGN KEY** — enforces referential integrity. Choose ON DELETE carefully:
- RESTRICT (default): safest — blocks delete if child rows exist.
- CASCADE: child rows deleted automatically — use only where children have no independent meaning.
- SET NULL: FK column set to NULL — requires nullable column.
- **UNIQUE** — allows NULL, multiple per table. Use for alternate identifiers (email, SKU).
- **CHECK** — rejects rows where condition is FALSE. NULL passes CHECK — add NOT NULL separately.
- **NOT NULL** — prevents empty columns. Combine with DEFAULT for required fields with fallbacks.
- **Name your constraints** — readable errors, easy ALTER TABLE DROP CONSTRAINT.
- **Table-level CHECK** — required when constraint spans multiple columns.
- **ALTER TABLE ADD CONSTRAINT** — validates all existing rows; clean data first.
- Constraints are the last line of defence — never remove them to make inserts easier.

✧ AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior database engineer. I just learned SQL constraints: PRIMARY KEY, FOREIGN KEY (with ON DELETE options), UNIQUE, CHECK, and NOT NULL. Give me 5 progressively harder schema design exercises. Start with a simple two-table schema and build up to a five-table system with cascades, composite PKs, and table-level CHECK constraints. For each exercise: show me an under-constrained schema, ask me to add all missing constraints, then show the correct solution and explain every decision. Include one common production mistake per exercise. Do not continue until I submit my answer.'

🔗 CHECKPOINT QUESTION

Final review — you are designing a **library** system. Tables needed: books (with ISBN as a unique identifier), members (with email unique), loans (a member borrows a book for a period). Apply everything from SQL 1.6: surrogate PKs, appropriate NOT NULL, UNIQUE on business identifiers, CHECK for logical rules (*due date after*

loandate, positive fine amounts), FK with correct ON DELETE for each relationship.
Write the complete CREATE TABLE statements with all constraints named.

✧ CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including sql 1.6 — constraints and schema design, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Order Management Schema — Fully Constrained ERD:** A detailed entity-relationship diagram showing four tables with all constraints ...